Python Zero to Hero by Building Games - Book 1 Companion Code

Repository README
Volume 1: Python Foundations and GUI Mini Games
Author: Pavlos Desopoulos
README version: 1.00

This repository is the official companion-code home for **Python Zero to Hero by Building Games, Volume 1**. It is designed to make the book's examples, exercises, debugging drills, and finished projects easy to inspect and run without forcing a beginner to reconstruct every long program by hand before seeing it work.

The repository contains two core companion resources:

1. **`python_zero_to_hero_code_launcher.py`** - a self-contained Tkinter launcher for browsing, running, stopping, searching, and exporting the code catalog.
2. **The complete Book 1 code archive (`.zip`)** - the extracted-file version of the companion code for readers who prefer to work directly in Thonny, IDLE, VS Code, another editor, or the terminal.

The launcher and the ZIP serve the same book, but they are useful in different ways. The launcher is best for fast inspection, comparison, and one-click execution. The ZIP is best when you want ordinary files that you can edit, rename, copy into your own practice folders, and run directly from an IDE.

> **Beginner recommendation:** use the book, launcher, and your own editor together. Type short examples yourself. Use the launcher to check finished examples quickly. Use the ZIP or the launcher's Export feature when you want editable files.

Contents

Quick start

Option A: use the launcher

1. Install **Python 3.10 or newer**. Python 3.10+ is the recommended baseline for the supplied launcher.
2. Make sure **Tkinter** works.
3. Download `python_zero_to_hero_code_launcher.py` from this repository.
4. Put the launcher in a normal folder where you have permission to create files.
5. Run it with Python.
6. Choose a chapter, choose a code type, select an item, and click **Run**.

Windows:

```
py --version
py -m tkinter
py python_zero_to_hero_code_launcher.py
```

If `py` is not available, try:

```
python --version
python -m tkinter
python python_zero_to_hero_code_launcher.py
```

macOS or Linux:

```
python3 --version
python3 -m tkinter
python3 python_zero_to_hero_code_launcher.py
```

Option B: use the ZIP archive

1. Download the `.zip` file from the repository.
2. Extract the whole archive to a normal folder. Do not work from inside the compressed ZIP window.
3. Open the extracted folder in Thonny or another Python editor.
4. Open the file you want to study.
5. Run the `.py` file with Python 3.
6. Keep all files of a multi-file project together in the same project folder.

What this repository is for

Book 1 moves from first Python statements through reusable program structure and then into Tkinter GUI projects. The companion repository exists to support that learning path, not replace it.

Use it to:

- check a working version after typing an example yourself;
- compare your file line by line with the book version;
- run longer projects without reconstructing them from several printed parts first;
- inspect solved exercises before attempting their twins;
- revisit a chapter's examples quickly;
- study deliberate Wrong / Right and debugging examples;
- run console programs and send them input from the launcher;
- launch later Tkinter programs in their own GUI windows;
- export one selected item or the complete code catalog to ordinary files;
- keep the printed book readable by allowing long projects to be split into explanatory sections while still providing a complete runnable version.

Large projects in the book are sometimes printed in parts for teaching clarity. Unless the lesson explicitly identifies a multi-file project, those printed parts belong to one Python file and should be assembled in order. The companion code gives you the complete version for checking.

Requirements

Required

- **Python 3**

- **Python 3.10+ recommended** for the current launcher build
- **Tkinter** for the launcher itself
- A writable folder for the launcher, because it creates run/export working directories

Not required for Book 1

- No external `pip` packages are required by the launcher.
- **Pygame is not required for Volume 1.** Pygame belongs to the later visual-game material, not this Book 1 launcher.

Editor choices

The launcher can be run directly with Python, so no editor is technically required. For learning and editing your own files, a beginner-friendly editor such as **Thonny** works well. IDLE is also suitable for early chapters. VS Code or another editor is fine if you already use it.

Repository files

A minimal repository release can be organized like this:

```
Python-Zero-To-Hero-Book-1/
|
|-- python_zero_to_hero_code_launcher.py
|-- <complete_book_1_code_archive>.zip
|-- README.md
|-- README.pdf        (optional reader-friendly copy)
`-- README.docx       (optional editable copy)
```

The exact ZIP filename may change between releases. The important distinction is simple:

- the **launcher** is the interactive browser/runner;
- the **ZIP** is the ordinary extracted code collection.

The launcher is self-contained and does **not** need the ZIP to be extracted before it can browse or run its embedded catalog.

Installing and checking Python

Windows

Install Python 3 from a trusted official source. If the installer offers **Add Python to PATH**, enable it. Then open Command Prompt or PowerShell and try:

```
py --version
```

or:

```
python --version
```

To test Tkinter:

```
py -m tkinter
```

or:

```
python -m tkinter
```

A small Tk window should open. Close it after the test.

macOS

Use Python 3 and test:

```
python3 --version
python3 -m tkinter
```

If the Tkinter test opens a small window, the GUI requirement is available.

Linux

Many Linux distributions separate Tkinter from the base Python package. Check first:

```
python3 --version
python3 -m tkinter
```

If Python reports that Tkinter is missing, install your distribution's Tkinter package. On some distributions this package is named `python3-tk`. Package names vary, so use the package appropriate to your Linux distribution.

Interpreter mismatch warning

If a package or module appears installed but your editor still reports `ModuleNotFoundError`, the editor may be using a different Python interpreter from the terminal. Check the interpreter selected by the editor before repeatedly reinstalling software.

Starting the launcher

The repository release should use this public-facing filename:

```
python_zero_to_hero_code_launcher.py
```

Run it from the folder that contains the file.

Windows:

```
py python_zero_to_hero_code_launcher.py
```

Alternative Windows command:

```
python python_zero_to_hero_code_launcher.py
```

macOS/Linux:

```
python3 python_zero_to_hero_code_launcher.py
```

You can also open the launcher file in Thonny and press **Run**.

Important: use a writable folder

At startup, the current launcher creates these folders in its current working directory:

```
PZTH_full_code_launcher_runs/
PZTH_full_code_launcher_exports/
```

If the folder is read-only or protected by the operating system, startup or later run/export actions may fail. Put the launcher in a normal user folder such as Documents, Desktop, or a project folder you own.

Launcher interface and workflow

The current launcher window is titled **PZTH Full Code Launcher**. Its layout is divided into selection controls on the left and code/output areas on the right.

1. Chapter selector

Choose the book chapter or appendix section you want to inspect.

2. Type selector

After choosing a chapter, the launcher shows the code categories that actually exist in that section. Typical categories include:

- Project
- Worked Example
- Example
- Solved Exercise
- Twin / Hint
- Wrong / Right
- Debug / Fix Drill
- Terminal
- File Tree

Not every chapter contains every type.

3. Search title/code

The search field filters the currently selected chapter and code type. It searches both the item's title and its displayed code text.

This means search is intentionally scoped. If you are in Chapter 7 and the Example type, a search does not scan every chapter in the whole book. Change the chapter/type first if you want a different area.

4. Item list

The item list contains the matching code blocks. A multi-file project is marked with **[multi-file]**.

5. Code panel

Selecting an item displays the book-aligned code in a large monospaced panel. The status line can also display notes about the item.

6. Run

Run writes the runnable version of the selected item into a timestamped run folder and starts it with the same Python interpreter that launched the launcher.

7. Stop

Stop asks the current child process to terminate. If it does not stop within about two seconds, the launcher escalates to killing that child process.

8. Export

Export saves the selected item to a folder you choose.

9. Export all code blocks

This exports the complete launcher catalog into a structured folder tree and writes an export manifest.

10. Console input

For programs that call `input()`, type your answer into the input field and press **Send input** or press Enter.

11. Output panel

Console output and error messages appear in the bottom output area. The launcher also prints start/stop/process-exit messages there.

Running console programs

Console examples are programs that print text in a terminal-style stream and may ask the user for input.

Typical workflow:

1. Select the chapter.
2. Select the type.
3. Select the code item.
4. Click **Run**.
5. Read the output area.
6. If the program asks for input, type the answer into **Input for console programs**.
7. Click **Send input** or press Enter.
8. Wait for the program to finish, or click **Stop** if it is intentionally long-running.

The launcher echoes submitted input into the output panel so the conversation with the console program remains visible.

If you click **Send input** when no console process is running, the launcher reports that no running process is waiting for input.

Running Tkinter GUI projects

From the Tkinter chapters, a selected program may open a **second window** of its own. This is expected.

The launcher remains open while the child GUI project runs. Interact with the project's window normally. When finished, close the project window. The launcher output area will eventually report the child process exit code.

If a GUI appears to have no console output, that is not necessarily a problem. A GUI program may communicate through buttons, labels, the Canvas, and other window elements instead of printing text.

If you need to stop a GUI project from the launcher, click **Stop**. If the project's window remains visible briefly, close that window as well.

Reference-only code blocks

Not every printed block in a programming book should run by itself. Some blocks are intentionally incomplete because they are:

- one fragment of a larger program;
- deliberate broken code in a Wrong / Right lesson;
- terminal output rather than Python source;
- a file-tree diagram;
- a debugging fragment designed to be read and repaired;
- a partial step that depends on code printed immediately before or after it.

The launcher preserves these blocks for fidelity to the book but marks them as **reference-only** when they should not be executed as standalone programs.

If you click Run on a reference-only item, the launcher shows an explanatory message rather than pretending the fragment is a complete program.

This distinction is important. A reference-only block being non-runnable is not automatically a defect. It can be the correct educational representation of the printed lesson.

Search and navigation

The launcher's navigation is intentionally simple:

```
Choose chapter -> choose code type -> optionally search -> choose item -> inspect/run/export
```

The search field checks:

- the item title;
- the displayed code.

The launcher automatically selects the first matching item after a chapter, type, or search change.

The default startup selection prefers **Chapter 1** and, when available, **Project**.

Exporting code

Export one selected item

Click **Export** and choose a destination folder. The launcher creates subfolders based on chapter/section and code type.

Conceptually, the exported structure looks like this:

```
<chosen folder>/
|
|-- ch01/
|   |-- project/
|   |   `-- <project file>.py
|   `-- example/
|       `-- <example file>.py
|
|-- ch15/
|   `-- project/
|       `-- <multi-file-project-name>/
|           |-- fighters.py
|           |-- moves.py
|           |-- battle.py
|           `-- main.py
`-- ...
```

For normal runnable items, the exported file contains the runnable version. For reference-only items, Export preserves the displayed reference content.

Export the complete catalog

Click **Export all code blocks** and choose a destination folder. The launcher exports every catalog item and creates:

```
PZTH_FULL_CODE_LAUNCHER_EXPORT_MANIFEST.txt
```

The manifest records the launcher title, author, number of exported items, and a short reminder about running Python and Tkinter GUI files.

Editing exported code

The recommended edit workflow is:

```
Export -> open the exported file in your editor -> edit -> save -> run from the editor
```

Important implementation detail: the code display panel is an inspector, not the authoritative source used by the Run button. The current launcher runs the embedded `runnable_code` stored in its catalog. Editing text you can see in the code panel does not rewrite that catalog entry. Export the code first if you want to make and run your own changes.

Using the ZIP archive directly

The ZIP is for readers who want ordinary files without using the launcher.

Extract first

Always extract the full archive before serious use. Running files from inside a compressed archive can produce confusing path and import problems, especially for projects that read files, create save data, or import sibling modules.

Preserve folders

Do not flatten the archive into one giant directory. Keep the supplied folder structure intact so that:

- imports can find neighboring modules;
- data files remain near the code that uses them;
- chapter organization stays understandable;
- multi-file projects keep their pieces together.

Run from the project folder

For a normal file:

```
python your_file.py
```

or:

```
python3 your_file.py
```

When using an editor, open the relevant extracted project folder so the working directory is predictable.

Multi-file project in Chapter 15

The current launcher catalog contains one explicitly multi-file project:

Mini Project (Solved): Multi-File Arena Duel

Its files are:

```
fighters.py
moves.py
battle.py
main.py
```

Keep all four in the same project folder and run:

```
python main.py
```

or:

```
python3 main.py
```

Do not run one helper module in isolation and assume that is the complete project.

Book and launcher structure

The companion code follows Book 1's progression.

Chapters 1 to 5: foundations

First scripts, printing, variables, input, conditions, loops, repetition, randomness, scores, and simple mechanics.

Chapters 6 to 10: core Python structures

Functions, lists, strings, nested lists/grids, dictionaries, inventories, player data, and richer game state.

Chapters 11 to 16: data, objects, files, and program structure

Advanced dictionaries, file saving, classes, object-oriented systems, modules, multi-file structure, menus, and game flow.

Chapters 17 to 24: Tkinter

Event-driven GUI programming, Rock Paper Scissors, quiz interfaces, Tic-Tac-Toe, a 21-card game, Reaction Challenge, Soccer Timer Duel, Cosmic Courier, Snake, and Fleet Battle.

Chapter 25 and appendices

Transition/reference material is represented where code blocks exist. Some sections naturally contain no runnable code and therefore may appear empty in the launcher.

Current launcher inventory

The README was generated against the supplied current launcher file and its embedded payload.

Measure	Current value
Launcher sections	31
Total catalog items	1450
Runnable items	929
Reference-only items	521
Projects	26
Multi-file projects	1

Item types

Type	Count
Example	984
Solved Exercise	191
Wrong / Right	139
Worked Example	77
Project	26
Debug / Fix Drill	17
File Tree	7
Twin / Hint	5
Terminal	4

A large catalog count is not the same thing as a large number of full programs. Many entries are individual examples, solved exercises, deliberate Wrong / Right comparisons, reference fragments, or pieces of larger lessons.

Project index

The current launcher identifies these project entries:

Chapter	Project	Multi-file
Chapter 1: Your First Python Steps	Mini Project (Solved): Hero Greeting Card	No
Chapter 2: Variables, Data and Player Interaction	Mini Project (Solved): Player Profile Terminal	No
Chapter 3: Decisions, Conditions and Game Logic	Mini Project (Solved): Relic Room Password Check	No
Chapter 4: Repetition, Loops and Doing Things More Than Once	Mini Project (Solved): ASCII Health Bar Animation	No
Chapter 5: Randomness, Score and Simple Mechanics	Mini Project (Solved): Slot Machine Simulator	No

Chapter	Project	Multi-file
Chapter 6: Functions, Reuse and Organizing Game Code	Mini Project (Solved): ASCII Boss Banner Generator	No
Chapter 7: Lists, Inventories and Storing Many Values	Mini Project (Solved): Adventurer Backpack	No
Chapter 8: Strings, Text Processing and Player Input Systems	Mini Project (Solved): Relic Hangman	No
Chapter 9: Nested Lists, Boards and Grid Thinking	Mini Project (Solved): Console Snake Game	No
Chapter 10: Dictionaries, Stats, Items and Rich Game Data	Mini Project (Solved): Pocket Monster Stat Battle	No
Chapter 11: Advanced Dictionaries and Game Databases	Mini Project (Solved): Data Quest Database	No
Chapter 12: Files, Saving and Persistent Progress	Mini Project (Solved): High Score Leaderboard	No
Chapter 13: Classes, Objects and Game Blueprints	Mini Project (Solved): Battle Card Builder	No
Chapter 14: Object-Oriented Game Systems	Mini Project (Solved): Pixel Pet Care	No
Chapter 15: Clean Code, Modules and Project Structure	Mini Project (Solved): Multi-File Arena Duel	Yes
Chapter 16: Menus, Game Flow and Player Experience	Mini Project (Solved): Cave Adventure Menu	No
Chapter 17: What Tkinter Is and When to Use It	Mini Project (Solved): Tkinter Tool Picker	No
Chapter 18: Build a Rock Paper Scissors GUI with Tkinter	Mini Project (Solved): Rock Paper Scissors GUI	No
Chapter 19: Build a Quiz Game GUI with Tkinter	Mini Project (Solved): Quiz Game GUI	No
Chapter 20: Build a Tic-Tac-Toe Grid with Tkinter	Mini Project (Solved): Tic-Tac-Toe Grid	No
Chapter 21: Build a Simple 21 Card Game with Tkinter	Mini Project (Solved): Simple 21 Card Game GUI	No
Chapter 22: Build Reaction Challenge and Soccer Timer Duel with Tkinter	Mini Project (Solved): Reaction Challenge GUI	No
Chapter 22: Build Reaction Challenge and Soccer Timer Duel with Tkinter	Soccer Timer Duel	No
Chapter 23: Cosmic Courier, a Polished Tkinter Arcade Game	Mini Project (Solved): Cosmic Courier Canvas Edition	No
Chapter 24: Final Tkinter Practice Games	Snake: A Real-Time Tkinter Practice Game	No
Chapter 24: Final Tkinter Practice Games	Fleet Battle: A Two-Board Tkinter Practice Game	No

The later chapters include the larger polished Tkinter games highlighted in the book, including **Cosmic Courier**, **Soccer Timer Duel**, **Snake**, and **Fleet Battle**.

Built-in audit mode

The launcher includes a non-GUI audit mode for release checking.

Run:

```
python python_zero_to_hero_code_launcher.py --audit
```

or:

```
python3 python_zero_to_hero_code_launcher.py --audit
```

The audit loads the embedded payload and uses Python's compiler to syntax-check each item marked runnable. For multi-file items, it checks each component file.

For the supplied build used to generate this README, the audit reports:

```
Title: PZTH Full Code Launcher
Author: Pavlos Desopoulos
Items: 1450
Projects: 26
Runnable items checked: 929
AUDIT PASSED
```

What the audit proves

It is useful for detecting syntax errors in code marked runnable.

What the audit does not prove

It does **not** execute all 929 runnable items. Therefore, it does not by itself prove that every interactive path, GUI behavior, file operation, timing behavior, or environment-dependent action works on every computer.

Treat `--audit` as a fast syntax/release check, not a replacement for runtime testing of important projects.

Generated folders and cleanup

When the launcher starts, it creates:

```
PZTH_full_code_launcher_runs/
PZTH_full_code_launcher_exports/
```

Run folder

Each run creates a timestamped subfolder. The selected code is written there before Python starts the child process. This gives the child program a real working directory and keeps generated files away from the launcher source.

A run folder name contains the item identifier and a timestamp.

Programs that create save files or other output may leave those files inside their run folder.

Export folder

The export folder is used as the default starting location when the file dialog asks where to save code. You are free to choose another destination.

Can these folders be deleted?

Yes, after the launcher and any child programs are closed, you can delete old run/export folders you no longer need. Check them first if you want to keep save files or exported practice code.

Troubleshooting

python or python3 is not recognized

Python is either not installed, not on your PATH, or the shell uses a different command.

Try:

```
py --version
python --version
python3 --version
```

Use whichever command reports a Python 3 version.

ModuleNotFoundError: No module named 'tkinter'

Tkinter is missing from that Python installation. On Windows/macOS, use a Python installation that includes Tcl/Tk. On Linux, install the Tkinter package for your distribution.

The launcher opens and immediately fails to create a folder

Move the launcher to a directory where you have write permission. The launcher creates its run and export directories in the current working directory.

The launcher says a block is reference-only

That block is intentionally not a standalone runnable program. Read it as part of the lesson, select a complete example/project, or export the surrounding code if you want to assemble a larger program.

The Run button does not use my edits from the visible code panel

That is expected in the current implementation. The Run button launches the embedded runnable version. Export the item, edit the exported file in your editor, then run your edited file there.

A program is waiting and nothing else happens

It may be waiting for `input()`. Read the output panel, type a response into the input field, and press **Send input**.

A Tkinter project opened another window

That is normal. The child project owns its own GUI window while the launcher remains open.

I cannot start a second program

The launcher allows one active child process at a time. Stop or close the current one before starting another.

Stop does not end the program instantly

The launcher first requests termination and waits briefly. If the process does not exit, it escalates to a kill. GUI cleanup can take a moment on some systems.

My file-reading example cannot find a file

File paths are relative to the program's working directory. With the launcher, runnable files are placed in their own run folder. With the ZIP, run the project from its extracted project folder and keep required data files beside the code as supplied.

The editor says a module is missing even though I installed it

Check which Python interpreter the editor is using. Installing into one Python environment does not automatically install into every Python environment on the computer.

Antivirus or security software warns about the launcher

The launcher creates files and starts child Python processes because that is how it runs the selected examples. Review the source, keep the repository from a trusted source, and use normal operating-system security practices. Do not disable security software merely to avoid a warning.

The GUI is too small

The launcher starts at approximately 1320 x 820 pixels and enforces a minimum size around 1100 x 680. Maximize the window or increase display resolution/scaling space if controls feel cramped.

Recommended beginner workflow

A strong learning loop is:

1. Read the explanation in the book.
2. Predict what the code should do.
3. Type the short example yourself.
4. Run your own file.
5. If it breaks, read the final traceback line first.
6. Compare your file with the launcher/book version.
7. Fix one cause at a time.
8. Run again.
9. Use Export when you need a clean complete copy.
10. Modify the exported copy to make it yours.

The launcher is most useful as a reference and verification tool. It should reduce pointless transcription failures without removing the practice of writing and debugging code yourself.

For teachers and parents

Volume 1 is designed for beginners and moves gradually from text-based Python into GUI projects.

A practical supervised workflow is:

- keep one folder per chapter;
- have the learner type short examples rather than copy everything;
- use the launcher to demonstrate the finished behavior before or after a lesson;
- use solved exercises as models and twins as independent adaptations;
- use Wrong / Right and Debug / Fix blocks to practice error recognition;
- export a clean file only after the learner has tried the task;
- keep personal information out of code, screenshots, filenames, usernames, and shared projects;
- install software only from trusted sources.

The launcher does not require external pip packages, which keeps Book 1 setup relatively small.

Reporting a problem

If you report a repository or launcher issue, include enough information to reproduce it.

Useful details:

```
Operating system:
Python version:
How Python was installed:
Launcher filename/version:
Chapter:
Code type:
Item title:
What you clicked or ran:
Expected result:
Actual result:
Full traceback or launcher output:
```

If the problem is visual, include a screenshot of the launcher or project window when appropriate.

Do **not** post passwords, API keys, private account data, personal file paths containing sensitive names, or other private information in a public issue report.

Technical notes for advanced users

Self-contained embedded catalog

The launcher stores its code catalog inside the Python launcher file as a Base64-encoded, zlib-compressed JSON payload. At startup it decodes and decompresses that payload in memory.

Practical consequence: the launcher can browse and run its catalog without reading the repository ZIP.

Do not hand-edit the large encoded payload unless you are deliberately rebuilding the launcher data. Human-facing maintenance is much safer when done at the source-data/generator level.

Child-process execution

Runnable items are started with:

```
<same Python interpreter> -u <generated script>
```

The `-u` flag keeps standard streams unbuffered, which makes console output appear promptly in the launcher.

The child process receives:

- its own generated working directory;
- piped standard input;
- piped standard output;
- standard error redirected into the same output stream.

Process isolation is not a security sandbox

The code runs in a **separate child process**, which is useful for stopping examples and preventing the launcher GUI from being the program itself. That is process isolation, not an operating-system security sandbox. A Python child process normally has the same user permissions as the launcher.

Run code only from trusted repository releases, especially if you modify or replace the embedded code.

Reference and runnable representations

Catalog items can carry separate displayed and runnable forms. This is what allows the launcher to preserve a printed educational fragment while also attaching a complete runnable equivalent where appropriate.

Important fields used by the implementation include concepts such as:

```
section_key
section_label
type
title
display_code
runnable_code
runnable
filename
files
main_file
notes
```

Multi-file items use a mapping of filenames to source text and identify the main entry file separately.

Export naming

Export paths are normalized to safe lowercase folder/file names for generated directory components. Chapter keys such as `ch01`, `ch15`, and appendix keys keep the export tree predictable.

Current multi-file support

The run/export logic creates all component files in one directory and executes the declared main file. In the current payload, the explicit multi-file project is the Chapter 15 arena project with `main.py` as its entry point.

Copyright and permitted educational use

The companion code is provided for learning and practice alongside **Python Zero to Hero by Building Games, Volume 1**. Readers may type, modify, and experiment with the book's examples for personal study, classroom use, and portfolio practice, consistent with the book's copyright and publishing notes.

This README does not invent or add an open-source license such as MIT, Apache, GPL, or BSD. If a repository release contains a separate `LICENSE` file, that file should be read together with the applicable book/repository copyright terms. If no separate license is supplied, do not assume that public GitHub visibility automatically grants unrestricted redistribution rights.

Product names, library names, and software names are used for educational identification. The book and companion repository are not presented as being affiliated with or endorsed by the creators of Python, Tkinter, Thonny, Pygame, or other tools mentioned in the lessons.

Frequently asked questions

Do I need the ZIP to run the launcher?

No. The current launcher contains its catalog internally. The ZIP is for direct access to ordinary code files.

Do I need the launcher to use the ZIP?

No. Extract the ZIP and run the files with Python 3 in your preferred editor.

Do I need Pygame?

No, not for Book 1. The launcher itself uses Tkinter, and the later Book 1 GUI projects use Tkinter.

Do I need to install anything with pip?

Not for the launcher. Its stated dependencies are Python 3 and Tkinter.

Why are some items not runnable?

Because some printed blocks are intentionally fragments, broken examples, terminal text, file trees, or other reference material. The launcher preserves them without pretending they are standalone programs.

Can I edit code inside the launcher and then click Run?

Not as an edit-run IDE workflow. Export the item and edit the exported file in Thonny, IDLE, VS Code, or another editor.

Where does the launcher put temporary run files?

Inside `PZTH_full_code_launcher_runs` in the working directory. Each run gets its own timestamped subfolder.

Where does Export put files?

You choose the destination folder. The launcher uses `PZTH_full_code_launcher_exports` as the initial suggested location.

Why does a project open a separate window?

Tkinter projects are standalone GUI programs, so they create their own windows.

How do I test the launcher without opening the GUI?

Use:

```
python python_zero_to_hero_code_launcher.py --audit
```

What should I do if the book shows a long project in several code blocks?

Unless the lesson explicitly says it is a multi-file project, assemble the pieces in order into one `.py` file. The launcher/companion archive provides the complete version for comparison.

Which Python version should I use?

Use a current Python 3 installation. The current launcher source recommends Python 3.10 or newer.

Final note

The companion repository is there to remove the least educational kind of friction: hunting for a missing character across a long printed project just to discover what the finished program is supposed to do. Use it as a reference, runner, and export tool, while still doing the valuable part yourself: reading code, predicting behavior, writing small sections, breaking things, debugging them, and changing the programs into your own versions.